SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

ENVIRONMENT SPECIFIC APPLICATION CONFIGURATION AND DEPLOYMENT USING ANSIBLE

One Playbook for Development Staging and Production

Detailed Project Report

Submitted by

Student 1 Name _____

Roll Number _____

Student 2 Name _____

Roll Number _____

Batch _____

Programme _____

Institution _____

DevOps Theory Internal Assessment

Abstract

Application settings often differ between development, staging, and production even when the application itself stays the same. Copying files and editing settings manually can introduce configuration drift, inconsistent behavior, and accidental use of development settings in production. This project demonstrates a repeatable way to manage those differences using Ansible.

The implementation uses one Ansible playbook, an inventory with three environment groups, and shared Jinja2 templates. A Campus Helpdesk web page provides a visible example. Each environment receives a different banner and Nginx error log level, while the development configuration enables an illustrative debug flag. The same templates generate the application page, a JSON configuration file, and a validated Nginx configuration.

Docker Compose supplies a Linux control node and three separate Linux managed nodes on one laptop. Ansible connects to the managed nodes over SSH, creates the application directory, installs the generated files, reloads Nginx through a change-triggered handler, and verifies the running environment. The practical demonstration includes a baseline deployment, an unchanged rerun, a check-mode preview, a production-only update, and another unchanged rerun.

Recorded validation confirmed successful deployment to all three nodes. The first baseline run reported five changed tasks per node, while the immediate repeat reported zero changed tasks for every node. The production update changed only the production host and left development and staging configuration unchanged. A repeat with the same update values again reported zero changes. The report explains the design, implementation, demonstration commands, measured results, troubleshooting, and the limits of the local container lab.

Keywords: Ansible, configuration management, inventory, environment variables, Jinja2, idempotence, handlers, Docker Compose, Nginx.

Table of contents

List of figures 5
List of tables 5
1 Introduction 6
1.1 Background 6
1.2 Problem statement 6
1.3 Objectives 6
1.4 Scope 6
2 Background and related concepts 7
2.1 Ansible and the control node 7
2.2 Inventory and environment groups 7
2.3 YAML and templates 7
2.4 Desired state and idempotence 7
2.5 Handlers and preview modes 7
3 Technology stack 8
4 System architecture 9
4.1 Control node and managed nodes 9
4.2 Environment settings 9
5 Application design 10
5.1 Campus Helpdesk configuration page 10
6 Containerisation and lab setup 11
6.1 Controller image 11
6.2 Managed-node image 11
6.3 SSH bootstrap and volumes 11
6.4 Platform requirements 11
7 Environment variables and templates 12
7.1 Shared settings and group variables 12
7.2 Generated configuration 12
7.3 Demonstration override 12
8 Ansible deployment implementation 13
8.1 Playbook tasks 13
8.2 Template validation and handlers 13

8.3 Health verification 13
9 Practical demonstration 14
9.1 Windows and macOS preparation 14
9.2 Confirm Ansible connectivity 14
9.3 Deploy all environment settings 14
9.4 Demonstrate an unchanged rerun 14
9.5 Preview the production update 15
9.6 Apply the scoped update 15
9.7 Repeat and restore 15
9.8 Work division for two presenters 15
10 Validation and results 16
10.1 Validation method 16
10.2 Interpretation 16
10.3 Evidence files 17
10.4 Practical limits of the results 17
11 Troubleshooting guide 18
12 Conclusion and future work 19
12.1 Conclusion 19
12.2 Limitations 19
12.3 Future work 19
References 20

List of figures and tables

List of figures

Figure 1 Inventory groups and managed nodes

Figure 2 Development page after baseline deployment

Figure 3 Production page after the scoped update

List of tables

Table 1 Core Ansible concepts

Table 2 Technology components and roles

Table 3 Verified runtime and image choices

Table 4 Environment settings

Table 5 Application endpoints

Table 6 Important project files

Table 7 Deployment tasks and change behavior

Table 8 Recorded validation results

Table 9 Common problems and fixes

1 Introduction

1.1 Background

A development environment supports frequent changes and detailed diagnostics. Staging provides a place to review configuration before it reaches users. Production needs settings appropriate for the deployed service. These environments can share application files while still requiring different configuration values. Keeping those values separate from deployment tasks makes the differences easier to review.

1.2 Problem statement

When operators copy and edit configuration files on each server, the files can gradually diverge. A setting may be changed on one host and forgotten on another. Teams may also duplicate deployment scripts for each environment, making later fixes harder to apply consistently. The project addresses this by keeping the requested settings in environment variable files and using one reusable deployment playbook.

1.3 Objectives

- Define development, staging, and production hosts in an Ansible inventory.
- Reuse one playbook and shared templates while applying different environment values.
- Validate server configuration before installation and reload Nginx only after a relevant change.
- Demonstrate an unchanged rerun, a preview, and an update limited to production.
- Record the resulting configuration and Ansible recap as evidence.

1.4 Scope

The implementation is a local educational lab. Three Linux containers represent environment roles and serve a static Campus Helpdesk page. The project focuses on configuration management and deployment of generated files. It does not implement a helpdesk backend, database, cloud deployment, continuous monitoring, or a CI pipeline.

2 Background and related concepts

2.1 Ansible and the control node

The Ansible control node connects to these Linux managed nodes over SSH and executes modules through Python. A persistent Ansible agent is not installed on the targets. This lab runs its controller in a Linux container [1].

Table 1 Core Ansible concepts

Concept	Role in this project
Inventory	Lists the managed hosts and groups them by environment.
Playbook	Defines the target hosts and an ordered list of tasks.
Module	Performs a task such as creating a directory or installing a template.
Variable	Supplies a value such as an environment name or log level.
Template	Uses placeholders to generate a file from those values.
Handler	Runs a related operation after a task reports a change.

2.2 Inventory and environment groups

The web parent group includes development, staging, and production. Each node belongs to one environment group and receives that group's inventory/group_vars settings. A command can target all web hosts or narrow execution to production [2].

2.3 YAML and templates

YAML expresses tasks and variable data. Jinja2 placeholders let one template produce different environment files. For example, the Nginx template receives info for development and warn for production. The template module compares the generated file with its destination before reporting changes [3].

2.4 Desired state and idempotence

Desired state describes the requested result, such as a directory or file with specific contents. Idempotence means repeating that request needs no additional changes. The file and template tasks support this behavior here, which the recap must confirm [4].

2.5 Handlers and preview modes

A changed Nginx template notifies its reload handler [5]. Check mode previews supported changes, while diff mode displays file differences. A preview does not replace verification after deployment [6].

3 Technology stack

Ansible is the configuration-management tool under study. Docker provides reproducible Linux lab machines, and Nginx serves the visible application. Keeping those roles separate helps explain which tool performs each operation.

Table 2 Technology components and roles

Component	Purpose
Ansible Core	Executes the inventory, playbook, modules, templates, and handlers.
YAML and Jinja2	Store structured settings and render shared configuration templates.
OpenSSH	Carries Ansible connections to the managed nodes.
Python	Runs Ansible on the controller and modules on managed nodes.
Nginx	Serves the HTML page, JSON configuration, and health response.
Docker Compose	Creates the local controller, managed nodes, network, and key volumes.

Table 3 Verified runtime and image choices

Item	Choice in this lab
Ansible Core	2.19.3, pinned in Dockerfile.controller.
Controller base	python:3.12-slim-bookworm; recorded runtime Python 3.12.15.
Managed-node base	debian:bookworm-slim with Python, SSH, sudo, curl, and Nginx.
Managed-node Python	/usr/bin/python3 from Debian Bookworm.
Docker engine	29.6.1 on the verified Windows laptop.
Browser access	Local host ports 8081, 8082, and 8083.

The Ansible package version is pinned for a repeatable demonstration. Base image tags can receive maintenance updates, so the build output and recorded runtime version identify what was actually used for this verification. A first build requires network access; the subsequent demo uses the locally built images.

4 System architecture

4.1 Control node and managed nodes

The controller reads the project from a read-only bind mount at /workspace. It reaches each managed node by its Compose service name on the project network. SSH connects as the ansible user, and sudo provides the privileges needed to write server files. The three target containers remain running after each temporary controller invocation finishes.



Figure 1 Inventory groups and managed nodes

Figure 1 is the output of ansible-inventory --graph from the tested project. Docker creates the containers and maps their web ports. Ansible then installs the environment configuration over SSH. The bootstrap helper prepares the lab SSH key before the targets start; it is setup infrastructure rather than the application deployment logic.

4.2 Environment settings

Table 4 Environment settings

Environment	Debug flag	Nginx log level	Host port
Development	Enabled	info	8081
Staging	Disabled	notice	8082
Production baseline	Disabled	warn	8083

Each container listens on port 80 internally. The host ports distinguish the browser pages. Compose owns these mappings, while the matching inventory values document the visible port on the page. All three nodes are containers on one laptop, not separate physical machines or public production servers.

5 Application design

5.1 Campus Helpdesk configuration page

The application is a small static web page that displays the environment banner and applied settings. It deliberately avoids backend complexity so the demonstration can focus on the configuration process. The page includes links to the deployed JSON file and server health response.

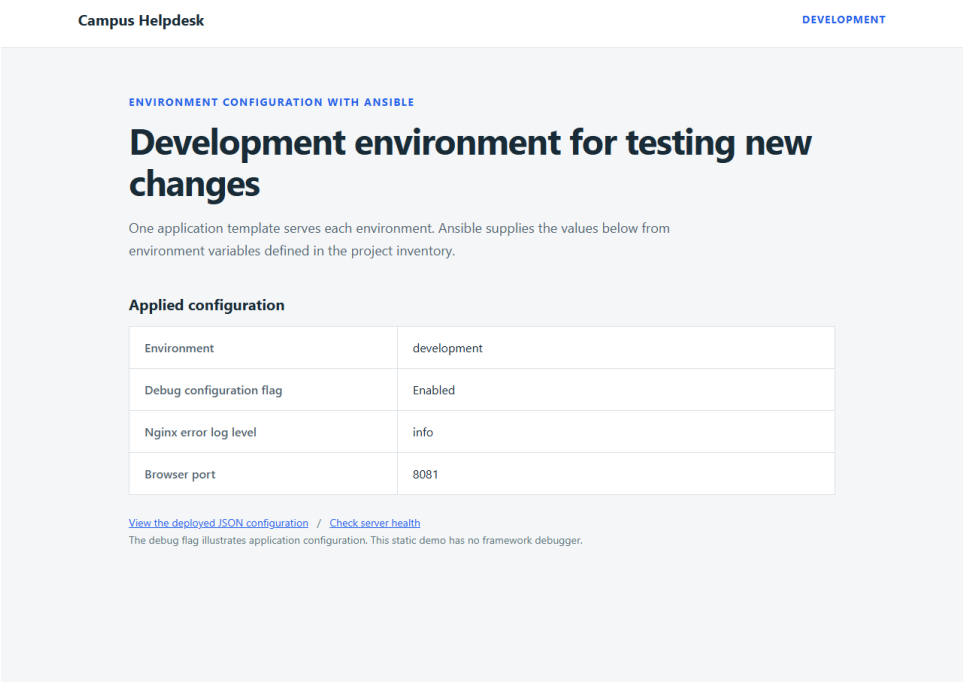


Figure 2 Development page after baseline deployment

Figure 2 renders HTML captured from the running development server. The debug value illustrates configuration and does not activate a framework debugger. The log level applies to Nginx itself. A no-store cache policy makes refreshes show current values, and HTTP headers identify the environment and log level for comparison with the JSON file.

Table 5 Application endpoints

Path	Response
/	HTML page with the environment banner and configuration.
/config.json	Generated application configuration as JSON.
/health	HTTP 200 JSON identifying the active environment.

6 Containerisation and lab setup

6.1 Controller image

Dockerfile.controller starts from a Python 3.12 Debian image, installs an SSH client, and installs ansible-core 2.19.3. ANSIBLE_CONFIG explicitly selects the project configuration. This matters for a Windows bind mount, where filesystem permissions may otherwise cause Ansible to ignore an automatically discovered configuration file.

```
FROM python:3.12-slim-bookworm
RUN pip install --no-cache-dir ansible-core==2.19.3
WORKDIR /workspace
ENV ANSIBLE_CONFIG=/workspace/ansible.cfg
```

6.2 Managed-node image

Dockerfile.node installs Nginx, OpenSSH Server, Python, sudo, and curl. A dedicated ansible user authenticates with the lab SSH key and can use sudo. The node startup helper starts SSH and keeps Nginx in the foreground. Nginx initially serves its default page until the playbook deploys the application.

6.3 SSH bootstrap and volumes

The bootstrap container creates a key pair if one does not already exist. The private key stays in a controller-only Docker volume. A separate volume provides the public authorized key to the targets. These generated key files do not enter the project source tree. The target nodes expose web ports only on the laptop loopback address; SSH stays inside the Compose network.

6.4 Platform requirements

Windows PowerShell and macOS Terminal use the same Docker Compose commands. Docker Desktop must provide a running Linux engine. No separate Ansible installation is required on the host. Docker Compose starts services and describes their relationships in compose.yaml [7].

```
docker compose up -d --build development staging production
docker compose ps
```

The first command builds and starts the lab. Use it before class because image and package downloads can take time. For later starts, omit --build when the images already exist.

7 Environment variables and templates

7.1 Shared settings and group variables

The all.yml group file supplies the shared application name, document root, and file owner. The environment group files provide the banner, debug flag, log level, page accent, and matching browser port. This separation lets reviewers see intended differences without reading duplicated deployment tasks.

```
app_environment: production
app_debug: false
app_log_level: warn
app_banner: Production environment ready for users
app_public_port: 8083
```

7.2 Generated configuration

The JSON template builds an object from these values and serializes it with to_nice_json. The HTML template uses escaping for text shown in the browser. The Nginx template applies the actual error-log level and serves the application directory. The same template source produces each environment's files.

```
error_log /var/log/nginx/error.log {{ app_log_level }};
root {{ app_root }};
add_header X-Lab-Environment "{{ app_environment }}" always;
```

7.3 Demonstration override

The prepared production-update.yml file sets a new banner and changes the log level to error. The demonstration passes it with -e and limits execution to the production group. Extra variables override the baseline group values for that invocation [8]. Repeating the baseline command without the override restores the baseline values; the demonstration therefore repeats the exact same override command when proving that the updated state is idempotent.

Table 6 Important project files

File or directory	Responsibility
inventory/hosts.ini	Hosts and environment membership.
inventory/group_vars/	Shared and environment settings.
playbooks/deploy.yml	Ordered deployment and verification tasks.
templates/	Reusable HTML, JSON, and Nginx file designs.
demo/production-update.yml	Production-only demonstration values.
evidence/	Recorded logs, responses, and verification summary.

8 Ansible deployment implementation

8.1 Playbook tasks

The playbook targets the web group and uses sudo. It disables fact gathering because the deployment needs no operating-system facts. Before writing files, an assertion checks that environment names, debug types, and log levels belong to the expected sets. The playbook then installs the shared templates and verifies the active server.

Table 7 Deployment tasks and change behavior

Task	Module	When it changes state
Validate settings	assert	Read-only validation.
Create application directory	file	Directory, ownership, or mode differs.
Deploy page and JSON	template	Rendered content or file attributes differ.
Deploy Nginx configuration	template	Validated configuration or attributes differ.
Reload Nginx	command handler	Nginx template reports a change.
Verify HTTP health	uri and assert	Read-only runtime checks.

8.2 Template validation and handlers

```
- name: Deploy and validate the Nginx configuration
  ansible.builtin.template:
    src: ../templates/nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    mode: "0644"
    validate: /usr/sbin/nginx -t -c %s
  notify: Reload Nginx
```

The template task tests the candidate configuration with nginx -t before installing it. The playbook flushes pending handlers before making HTTP checks, so the checks see the newly applied environment. The reload command runs only after a relevant configuration change; it is not an unconditional command in every run.

8.3 Health verification

After a real deployment, the uri task requests /health on each managed node. It retries briefly while a reload takes effect, and the following assertion confirms the environment identity. The preview skips these runtime tasks because check mode does not install the proposed files. The play recap separately reports failures and unreachable hosts.

9 Practical demonstration

9.1 Windows and macOS preparation

On Windows, open PowerShell in D:\ansible-theory-ia. On macOS, open Terminal in the copied project folder, for example ~/ansible-theory-ia. Start Docker Desktop and wait for its Linux engine. docker version should show both Client and Server information. The remaining commands are the same on both platforms.

```
docker version
docker compose version
docker compose up -d --build development staging production
```

9.2 Confirm Ansible connectivity

```
docker compose run --rm controller ansible web -m ansible.builtin.ping
```

The command should return SUCCESS and pong for all three hosts. This module check runs over SSH and confirms that Ansible can execute Python on the targets. It is different from an ICMP network ping. The Docker prefix starts a temporary Linux controller; ansible is the actual tool command that follows it.

9.3 Deploy all environment settings

```
docker compose run --rm controller ansible-playbook playbooks/deploy.yml
```

After the run, open localhost:8081, localhost:8082, and localhost:8083. Compare the labels and values with Table 4, then open /config.json on each page. On fresh containers the deployment writes the files and reloads Nginx. The next run compares those requested files with the existing state.

9.4 Demonstrate an unchanged rerun

```
docker compose run --rm controller ansible-playbook playbooks/deploy.yml
```

Point to changed=0 for all hosts in PLAY RECAP. Explain that Ansible checked the requested state and found matching files. This observation supports idempotence for this playbook under the demonstrated conditions; it does not establish that every arbitrary shell task is safe to repeat.

Practical demonstration continued

9.5 Preview the production update

```
docker compose run --rm controller ansible-playbook playbooks/deploy.yml --limit
production -e "@demo/production-update.yml" --check --diff
```

The limit selects only production. The override file supplies the new values, and check/diff mode displays the proposed file changes. Refreshing the live production page should still show its baseline banner. The preview is useful evidence that a predicted change and an applied change are different operations.

9.6 Apply the scoped update

```
docker compose run --rm controller ansible-playbook playbooks/deploy.yml --limit
production -e "@demo/production-update.yml"
```

Refresh localhost:8083. Its banner becomes Production settings updated using Ansible, and its log level becomes error. Development and staging keep their baseline values. The recap includes only prod-server, and the Nginx handler runs because the server configuration changed.

9.7 Repeat and restore

Run the exact update command again to show changed=0 for production. Including the same override matters because that file defines the desired updated state. To prepare for the next rehearsal, run the baseline playbook without the override. It restores production to warn and the original banner. Stop the lab afterward using the project-scoped down command.

```
docker compose run --rm controller ansible-playbook playbooks/deploy.yml
docker compose down
```

9.8 Work division for two presenters

Presenter 1 can introduce the configuration problem and explain the inventory, variables, and templates. Presenter 2 can run the baseline, preview, and update while explaining the recap and handler. Both presenters should rehearse the commands and describe their actual contributions accurately.

10 Validation and results

10.1 Validation method

The local validation ran on 19 October 2026 using the built container images and Ansible Core 2.19.3. It checked SSH connectivity, playbook syntax, deployment, repeated execution, check mode, and the scoped production update. HTTP verification read the generated JSON and health endpoints and compared response headers with the deployed settings. Full command output is retained in evidence/*.txt, and evidence/results.json records the run summary.

Table 8 Recorded validation results

Operation	Observed change count	Outcome
First baseline deployment	5 on each of 3 hosts	All environments deployed.
Baseline repeated	0 on each of 3 hosts	Existing requested state retained.
Production preview	Proposed changes only	Live configurations unchanged.
Production update	4 on prod-server only	Banner and log level updated.
Production update repeated	0 on prod-server	Updated state retained.
Baseline restored	4 on production, 0 elsewhere	Lab ready for another demo.

10.2 Interpretation

The changed count includes operations, not the number of settings or servers. The first deployment reports five changed tasks because it creates the directory, writes three files, and runs the Nginx reload handler. The production update reports four because it changes three generated files and runs the handler. All recorded deployment runs finished with failed=0 and unreachable=0. The second runs reported no changes, and the reload handler did not execute.

The production preview showed file differences while HTTP configuration comparisons remained identical to the baseline. After applying the update, development and staging JSON stayed unchanged. Production response headers agreed with its new log level. The validation then restored the baseline so a new presentation can begin from a consistent starting point.

Validation evidence

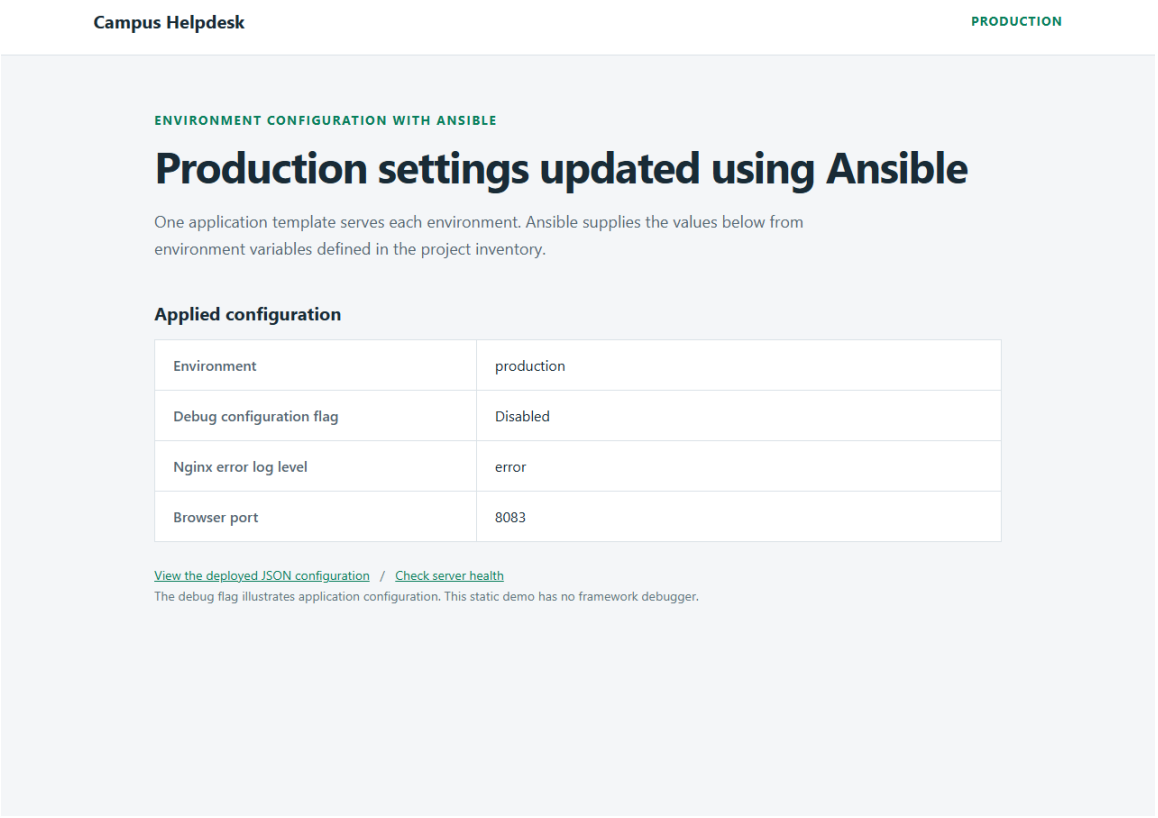


Figure 3 Production page after the scoped update

Figure 3 renders HTML captured from the running production server after the update. The banner and error-log level reflect the override file. This rendering is recorded evidence from verification, rather than a substitute for the live browser refresh during the presentation.

10.3 Evidence files

The baseline and updated JSON responses are stored separately so the environment differences can be inspected. The preview log records the proposed changes. The repeated-run logs show the unchanged recaps. Keeping those records makes the demonstration reviewable without inventing screenshots or fixed output values.

10.4 Practical limits of the results

These checks establish the demonstrated local behavior. They do not measure performance, prove availability during every possible failure, or validate deployment to remote cloud infrastructure. macOS uses the same Compose commands and Linux images, but the recorded execution is from the Windows laptop. A different laptop still needs Docker Desktop, available host ports, and enough resources to build and run the lab.

11 Troubleshooting guide

Table 9 Common problems and fixes

Problem	Likely cause	Action
Cannot reach Docker engine	Docker Desktop or Linux engine stopped.	Open Docker Desktop and wait for Server information in docker version.
Browser port occupied	Another program uses 8081, 8082, or 8083.	Close the conflict or change its Compose host mapping and matching display variable.
Host UNREACHABLE	Target not healthy or SSH key setup incomplete.	Use docker compose ps and inspect target/bootstrap logs.
Nginx template fails validation	Invalid server syntax or unexpected variable value.	Read the failed task output and fix the template or group variable.
Preview leaves browser unchanged	Command used --check.	Apply the same update without --check --diff.
Baseline resets production	Command omitted the update override.	Repeat the same override when demonstrating the updated desired state.
Page shows old configuration	Browser was not refreshed or wrong port opened.	Refresh the correct environment page and inspect /config.json.
A repeat reports changed tasks	Requested values or deployed files differ.	Read the diff and compare the exact command and override values.

Use project-scoped diagnostics instead of broad cleanup. docker compose logs development staging production shows the target logs, and docker compose down stops this lab. Avoid removing unrelated containers, images, or volumes when troubleshooting the demonstration.

12 Conclusion and future work

12.1 Conclusion

The project demonstrates how one Ansible playbook can deploy a shared application with different environment settings. Inventory groups select the hosts, group variables supply the differences, and Jinja2 templates generate the files. Nginx configuration validation and a change-triggered reload handler connect file deployment with the running service.

Recorded runs showed that the baseline deployment configured every environment, unchanged reruns made zero changes, check mode previewed a production update without applying it, and the actual update affected production alone. This provides a clear practical example of configuration management for a theory assessment.

12.2 Limitations

- All environments are containers on one laptop and share its resources and failure domain.
- The application is static and has no database or end-user transaction flow.
- The debug flag illustrates configuration rather than enabling a framework debugger.
- Ansible corrects this configuration when invoked; no continuous reconciliation service runs in the background.
- The disposable lab disables SSH host-key checking and grants broad sudo to its lab user. A real deployment needs verified host keys and appropriately scoped credentials.
- The demonstration does not include secret management, automated rollback, or production monitoring.

12.3 Future work

The next step is to replace container hostnames with real server addresses and adapt service management to those operating systems. Ansible Vault can protect sensitive values, while roles can package the deployment for reuse across more applications. A real backend, automated tests of configuration values, and monitoring after deployment would extend the project beyond the local demonstration. These are proposed extensions and are not implemented in the current lab.

References

[1] Ansible Community, Installing Ansible and control node requirements. Accessed 19 October 2026. https://docs.ansible.com/projects/ansible/latest/installation_guide/intro_installation.html

[2] Ansible Community, How to build your inventory. Accessed 19 October 2026. https://docs.ansible.com/projects/ansible/latest/inventory_guide/intro_inventory.html

[3] Ansible Community, ansible.builtin.template module. Accessed 19 October 2026. https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/template_module.html

[4] Ansible Community, Ansible playbooks and idempotence. Accessed 19 October 2026. https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html

[5] Ansible Community, Handlers running operations on change. Accessed 19 October 2026. https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_handlers.html

[6] Ansible Community, Validating tasks with check and diff modes. Accessed 19 October 2026. https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_checkmode.html

[7] Docker, Docker Compose documentation. Accessed 19 October 2026. https://docs.docker.com/compose/

[8] Ansible Community, Using variables and variable precedence. Accessed 19 October 2026. https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_variables.html

[9] Nginx, Beginner guide. Accessed 19 October 2026. https://nginx.org/en/docs/beginners_guide.html

Implementation evidence: the playbook, inventory, templates, Compose files, and recorded logs in this project. No external source repository is claimed for this implementation.